



ĐẠI HỌC HUẾ  
TRƯỜNG ĐẠI HỌC KHOA HỌC

# Phát triển ứng dụng IoT

## Nhóm 3

*Đề tài: Xây dựng đồng hồ thời gian thực với  
ESP32 và NTP*

ĐH. KHOA HỌC, NĂM HỌC 2025-2026.2

# Lời mở đầu

Trong những năm gần đây, sự phát triển mạnh mẽ của công nghệ thông tin và truyền thông đã thúc đẩy sự ra đời và phát triển của Internet of Things (IoT). IoT cho phép các thiết bị vật lý kết nối với nhau thông qua Internet, từ đó tạo nên các hệ thống thông minh có khả năng thu thập, xử lý và trao đổi dữ liệu một cách tự động.

Một trong những yếu tố quan trọng trong các hệ thống IoT là khả năng quản lý và đồng bộ thời gian chính xác. Thời gian đóng vai trò then chốt trong việc đảm bảo các hoạt động diễn ra đúng trình tự và theo kế hoạch. Việc sử dụng giao thức NTP (Network Time Protocol) giúp các thiết bị có thể đồng bộ thời gian với máy chủ chuẩn trên Internet, từ đó nâng cao độ chính xác và tính tin cậy của hệ thống.

Bên cạnh đó, việc tích hợp các nền tảng giao tiếp như Telegram vào hệ thống IoT giúp người dùng có thể dễ dàng tương tác và điều khiển thiết bị từ xa thông qua các câu lệnh đơn giản. Điều này không chỉ tăng tính tiện lợi mà còn mở ra nhiều hướng phát triển cho các ứng dụng thông minh trong thực tế.

Xuất phát từ những yêu cầu và xu hướng trên, đề tài “Xây dựng đồng hồ thời gian thực với ESP32 và NTP” được thực hiện nhằm xây dựng một hệ thống có khả năng đồng bộ thời gian qua Internet, hiển thị thời gian thực và gửi thông báo theo lịch trình thông qua Telegram. Thông qua đề tài, sinh viên có cơ hội vận dụng các kiến thức đã học về lập trình IoT, giao tiếp mạng và phát triển ứng dụng IoT vào một bài toán cụ thể.

Nội dung của tiểu luận được trình bày gồm các phần chính: Cơ sở lý thuyết, Phân tích hệ thống, Thiết kế hệ thống, Triển khai và kết quả và Kết luận.

# Mục Lục

|                                            |          |
|--------------------------------------------|----------|
| <b>Phát triển ứng dụng IoT</b>             | <b>0</b> |
| <b>Lời mở đầu</b>                          | <b>1</b> |
| <b>Mục Lục</b>                             | <b>2</b> |
| <b>CHƯƠNG 1: GIỚI THIỆU</b>                | <b>4</b> |
| 1.1. Giới thiệu tổng quan về đề tài        | 4        |
| 1.2. Mục tiêu đề tài                       | 4        |
| 1.3. Phạm vi nghiên cứu                    | 5        |
| <b>CHƯƠNG 2: CƠ SỞ LÝ THUYẾT</b>           | <b>5</b> |
| 2.1. Tổng quan về Internet of Things (IoT) | 5        |
| 2.2. Vi điều khiển ESP32                   | 6        |
| 2.3. Giao thức NTP (Network Time Protocol) | 6        |
| 2.4. Màn hình OLED SSD1306                 | 6        |
| 2.5. Giao thức HTTP/HTTPS                  | 7        |
| 2.6. Telegram Bot và Telegram Bot API      | 7        |
| 2.7. Tổng kết chương                       | 8        |
| <b>CHƯƠNG 3: PHÂN TÍCH HỆ THỐNG</b>        | <b>8</b> |
| 3.1. Tổng quan hệ thống                    | 8        |
| 3.2. Yêu cầu hệ thống                      | 8        |
| 3.2.1. Yêu cầu chức năng                   | 8        |
| 3.2.2. Yêu cầu phi chức năng               | 9        |
| 3.3. Sơ đồ khối hệ thống                   | 9        |
| 3.4. Mô tả hoạt động hệ thống              | 10       |
| 3.5. Use Case hệ thống                     | 10       |
| 3.6. Kịch bản hoạt động (Scenario)         | 11       |
| Kịch bản 1: Xem thời gian                  | 11       |

|                                        |           |
|----------------------------------------|-----------|
| Kịch bản 2: Đặt lịch thông báo         | 11        |
| 3.7. Tổng kết chương                   | 11        |
| <b>CHƯƠNG 4: THIẾT KẾ HỆ THỐNG</b>     | <b>12</b> |
| 4.1. Tổng quan thiết kế                | 12        |
| 4.2. Thiết kế phần cứng                | 12        |
| 4.2.1. Danh sách linh kiện             | 12        |
| 4.2.2. Sơ đồ kết nối phần cứng         | 12        |
| 4.2.3. Nguyên lý hoạt động phần cứng   | 14        |
| 4.3. Thiết kế phần mềm                 | 14        |
| 4.3.1. Kiến trúc chương trình          | 14        |
| 4.3.2. Thiết kế các chức năng chính    | 15        |
| a) Kết nối WiFi                        | 15        |
| b) Đồng bộ thời gian NTP               | 15        |
| c) Hiển thị thời gian trên OLED        | 15        |
| d) Giao tiếp với Telegram              | 15        |
| e) Xử lý lệnh từ người dùng            | 16        |
| f) Logic gửi thông báo                 | 16        |
| 4.3.3. Chương trình điều khiển         | 16        |
| 4.4. Lưu đồ thuật toán                 | 25        |
| 4.5. Tổng kết chương                   | 27        |
| <b>CHƯƠNG 5: TRIỂN KHAI VÀ KẾT QUẢ</b> | <b>27</b> |
| 5.1. Môi trường phát triển             | 27        |
| 5.2. Triển khai hệ thống               | 27        |
| 5.2.1. Kết nối WiFi                    | 27        |
| 5.2.2. Đồng bộ thời gian NTP           | 28        |
| 5.2.3. Hiển thị thời gian trên OLED    | 28        |
| 5.2.4. Tích hợp Telegram Bot           | 28        |
| 5.2.5. Xử lý lệnh từ người dùng        | 29        |

|                                    |           |
|------------------------------------|-----------|
| 5.2.6. Chức năng thông báo tự động | 30        |
| 5.3. Kết quả đạt được              | 30        |
| 5.4. Đánh giá hệ thống             | 30        |
| 5.4.1. Ưu điểm                     | 30        |
| 5.4.2. Hạn chế                     | 31        |
| 5.5. Hướng phát triển              | 31        |
| 5.6. Tổng kết chương               | 31        |
| <b>CHƯƠNG 6: KẾT LUẬN</b>          | <b>31</b> |
| 6.1. Kết luận                      | 31        |
| 6.2. Hướng phát triển              | 32        |
| 6.3. Tổng kết                      | 32        |
| <b>Tài liệu tham khảo</b>          | <b>33</b> |

# CHƯƠNG 1: GIỚI THIỆU

## 1.1. Giới thiệu tổng quan về đề tài

Trong bối cảnh công nghệ phát triển mạnh mẽ, Internet of Things (IoT) đã trở thành một xu hướng quan trọng, cho phép các thiết bị vật lý kết nối và trao đổi dữ liệu với nhau thông qua Internet. Nhờ đó, các hệ thống thông minh có thể được xây dựng nhằm nâng cao hiệu quả trong quản lý, giám sát và tự động hóa.

Một trong những thành phần quan trọng trong nhiều hệ thống IoT là khả năng quản lý và xử lý thời gian chính xác. Đồng hồ thời gian thực (Real-Time Clock) đóng vai trò cốt lõi trong việc đảm bảo các hoạt động diễn ra đúng thời điểm. Tuy nhiên, thay vì sử dụng các module phần cứng RTC truyền thống, việc đồng bộ thời gian từ máy chủ NTP (Network Time Protocol) thông qua Internet giúp hệ thống đạt được độ chính xác cao hơn và giảm chi phí phần cứng.

Bên cạnh đó, sự phát triển của các nền tảng nhắn tin như Telegram cho phép tích hợp các chức năng điều khiển và thông báo từ xa một cách thuận tiện. Việc sử dụng Telegram Bot giúp người dùng có thể tương tác với hệ thống IoT thông qua các câu lệnh đơn giản, từ đó nâng cao tính linh hoạt và khả năng mở rộng của hệ thống.

Xuất phát từ những lý do trên, đề tài “Xây dựng đồng hồ thời gian thực với ESP32 và NTP” được thực hiện nhằm xây dựng một hệ thống IoT hoàn chỉnh, có khả năng hiển thị thời gian thực, đồng bộ thời gian qua Internet và gửi thông báo theo lịch trình thông qua Telegram.

## 1.2. Mục tiêu đề tài

Mục tiêu chính của đề tài là thiết kế và xây dựng một hệ thống đồng hồ thời gian thực sử dụng vi điều khiển ESP32, kết hợp với các công nghệ mạng và dịch vụ trực tuyến. Cụ thể, đề tài hướng tới các mục tiêu sau:

- Xây dựng hệ thống có khả năng kết nối WiFi và truy cập Internet

- Đồng bộ thời gian thực từ máy chủ NTP để đảm bảo độ chính xác
- Hiển thị thời gian thực trên màn hình OLED thông qua giao tiếp I2C
- Xây dựng Telegram Bot để giao tiếp với người dùng
- Cho phép người dùng yêu cầu xem thời gian hiện tại thông qua lệnh `/time`
- Cho phép người dùng thiết lập lịch thông báo thông qua lệnh `/setupnoti`
- Tự động gửi thông báo qua Telegram khi đến thời gian đã cài đặt

Thông qua việc thực hiện các mục tiêu trên, đề tài giúp củng cố kiến thức về lập trình IoT, giao tiếp mạng và phát triển ứng dụng IoT thực tế.

### 1.3. Phạm vi nghiên cứu

Đề tài tập trung vào việc thiết kế và triển khai một hệ thống IoT ở mức cơ bản đến trung bình, sử dụng nền tảng ESP32 và môi trường giả lập Wokwi. Các nội dung chính được nghiên cứu và thực hiện bao gồm:

- Kết nối và cấu hình WiFi trên ESP32
- Sử dụng giao thức NTP để đồng bộ thời gian từ máy chủ Internet
- Giao tiếp với màn hình OLED SSD1306 thông qua giao thức I2C
- Giao tiếp với Telegram thông qua Telegram Bot API sử dụng HTTP/HTTPS
- Xây dựng logic xử lý lệnh và điều khiển hệ thống từ xa
- Thiết lập và thực hiện chức năng gửi thông báo theo lịch trình

Đề tài không đi sâu vào các vấn đề nâng cao như bảo mật hệ thống, tối ưu hiệu năng, hay triển khai trên phần cứng thực tế với quy mô lớn. Ngoài ra, việc lưu trữ dữ liệu lâu dài (ví dụ EEPROM hoặc cơ sở dữ liệu) cũng không nằm trong phạm vi của đề tài.

Tuy nhiên, hệ thống được xây dựng có thể làm nền tảng để phát triển các ứng dụng IoT phức tạp hơn trong tương lai.

# CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

## 2.1. Tổng quan về Internet of Things (IoT)

Internet of Things (IoT) là mô hình kết nối các thiết bị vật lý thông qua Internet, cho phép chúng thu thập, trao đổi và xử lý dữ liệu một cách tự động. Các thiết bị trong hệ thống IoT có thể bao gồm cảm biến, vi điều khiển, thiết bị truyền thông và các nền tảng phần mềm.

Một hệ thống IoT cơ bản thường gồm ba thành phần chính:

- **Thiết bị (Things):** Là các phần cứng như vi điều khiển, cảm biến, thiết bị hiển thị
- **Kết nối (Connectivity):** Cho phép thiết bị truyền dữ liệu thông qua WiFi, Bluetooth hoặc mạng di động
- **Xử lý và ứng dụng (Processing & Application):** Xử lý dữ liệu và cung cấp giao diện cho người dùng

Trong đề tài này, hệ thống IoT được xây dựng bao gồm ESP32 (thiết bị), WiFi (kết nối) và Telegram (ứng dụng giao tiếp với người dùng).

## 2.2. Vi điều khiển ESP32

ESP32 là một vi điều khiển tích hợp WiFi và Bluetooth, được phát triển bởi hãng Espressif Systems. Đây là một trong những nền tảng phổ biến trong các ứng dụng IoT nhờ vào hiệu năng cao, chi phí thấp và khả năng kết nối mạng mạnh mẽ.

Một số đặc điểm nổi bật của ESP32:

- Tích hợp WiFi và Bluetooth
- Tốc độ xử lý cao (lên đến 240MHz)
- Nhiều chân GPIO hỗ trợ các giao thức như I2C, SPI, UART
- Tiêu thụ điện năng thấp



Trong đề tài, ESP32 đóng vai trò trung tâm, thực hiện các chức năng:

- Kết nối WiFi
- Đồng bộ thời gian từ NTP
- Giao tiếp với màn hình OLED
- Gửi và nhận dữ liệu từ Telegram

## 2.3. Giao thức NTP (Network Time Protocol)

NTP (Network Time Protocol) là một giao thức mạng được sử dụng để đồng bộ thời gian giữa các thiết bị trong hệ thống máy tính thông qua Internet. NTP giúp đảm bảo các thiết bị có cùng thời gian chính xác, với sai số rất nhỏ.

Nguyên lý hoạt động của NTP:

- Thiết bị gửi yêu cầu đến máy chủ NTP
- Máy chủ trả về thời gian chuẩn (UTC)
- Thiết bị tính toán và điều chỉnh thời gian nội bộ

Trong đề tài, ESP32 sử dụng thư viện NTPClient để kết nối tới máy chủ NTP (ví dụ: pool.ntp.org) và lấy thời gian thực theo múi giờ Việt Nam (GMT+7).

Việc sử dụng NTP giúp loại bỏ nhu cầu sử dụng module RTC phần cứng, đồng thời đảm bảo độ chính xác cao và cập nhật liên tục.

## 2.4. Màn hình OLED SSD1306

OLED SSD1306 là loại màn hình hiển thị phổ biến trong các dự án IoT nhờ kích thước nhỏ gọn, tiêu thụ điện năng thấp và khả năng hiển thị rõ nét.

Một số đặc điểm của OLED SSD1306:

- Độ phân giải phổ biến: 128x64 pixel
- Giao tiếp thông qua I2C
- Không cần đèn nền
- Hiển thị tốt trong môi trường ánh sáng yếu

Trong hệ thống, màn hình OLED được sử dụng để hiển thị thời gian thực do ESP32 lấy từ máy chủ NTP. Giao tiếp giữa ESP32 và OLED được thực hiện thông qua hai chân:

- SDA (GPIO21)
- SCL (GPIO22)

## 2.5. Giao thức HTTP/HTTPS

HTTP (HyperText Transfer Protocol) là giao thức truyền tải dữ liệu giữa client và server trên Internet. HTTPS là phiên bản bảo mật của HTTP, sử dụng mã hóa SSL/TLS để bảo vệ dữ liệu.

Trong đề tài, ESP32 sử dụng giao thức HTTPS để:

- Gửi yêu cầu đến Telegram Bot API
- Nhận dữ liệu phản hồi từ server

Việc sử dụng HTTPS giúp đảm bảo tính bảo mật và toàn vẹn dữ liệu khi truyền qua mạng.

## 2.6. Telegram Bot và Telegram Bot API

Telegram Bot là một tài khoản đặc biệt trên nền tảng Telegram, cho phép lập trình viên xây dựng các ứng dụng tự động tương tác với người dùng thông qua tin nhắn.

Telegram cung cấp Bot API để:

- Gửi tin nhắn
- Nhận tin nhắn từ người dùng
- Xử lý các lệnh

Trong đề tài, Telegram Bot được sử dụng để:

- Nhận lệnh từ người dùng như `/time`, `/setupnoti`
- Gửi phản hồi và thông báo theo thời gian thực

## 2.7. Tổng kết chương

Chương này đã trình bày các cơ sở lý thuyết liên quan đến đề tài, bao gồm tổng quan về IoT, vi điều khiển ESP32, giao thức NTP, màn hình OLED, giao thức HTTP/HTTPS và Telegram Bot.

Những kiến thức này là nền tảng quan trọng để triển khai hệ thống đồng hồ thời gian thực có khả năng kết nối Internet và tương tác với người dùng thông qua Telegram, sẽ được trình bày chi tiết trong các chương tiếp theo.

# CHƯƠNG 3: PHÂN TÍCH HỆ THỐNG

## 3.1. Tổng quan hệ thống

Hệ thống được xây dựng trong đề tài là một ứng dụng Internet of Things (IoT) có chức năng chính là hiển thị thời gian thực và gửi thông báo theo lịch trình thông qua nền tảng Telegram. Hệ thống sử dụng vi điều khiển ESP32 làm trung tâm xử lý, kết nối Internet thông qua WiFi để đồng bộ thời gian từ máy chủ NTP và giao tiếp với Telegram Bot API.

Các chức năng chính của hệ thống bao gồm:

- Hiển thị thời gian thực trên màn hình OLED
- Nhận lệnh từ người dùng thông qua Telegram
- Xử lý các lệnh như `/time` và `/setupnoti`
- Gửi thông báo tự động khi đến thời gian đã cài đặt

Hệ thống cho phép người dùng tương tác từ xa một cách thuận tiện, không cần can thiệp trực tiếp vào phần cứng.

## 3.2. Yêu cầu hệ thống

### 3.2.1. Yêu cầu chức năng

Hệ thống cần đáp ứng các yêu cầu chức năng sau:

- Kết nối WiFi để truy cập Internet
- Đồng bộ thời gian thực từ máy chủ NTP
- Hiển thị thời gian thực trên màn hình OLED
- Nhận và xử lý lệnh từ Telegram
- Hỗ trợ lệnh `/time` để trả về thời gian hiện tại
- Hỗ trợ lệnh `/setupnoti HH:MM` để thiết lập thời gian thông báo
- Gửi thông báo tự động qua Telegram khi đến thời gian đã đặt

### 3.2.2. Yêu cầu phi chức năng

Ngoài các yêu cầu chức năng, hệ thống cần đảm bảo các yêu cầu phi chức năng sau:

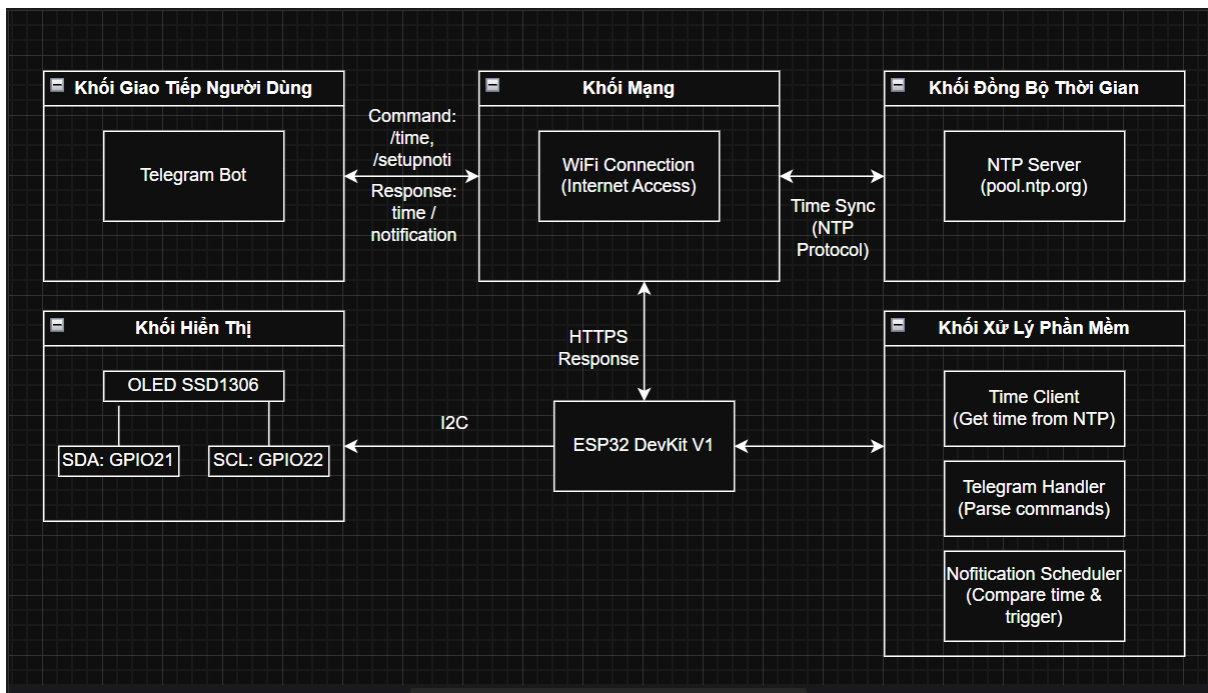
- **Độ chính xác:** Thời gian phải được đồng bộ chính xác thông qua NTP
- **Tính ổn định:** Hệ thống hoạt động liên tục, ít xảy ra lỗi
- **Khả năng phản hồi:** Thời gian phản hồi lệnh từ Telegram nhanh
- **Tính đơn giản:** Giao diện sử dụng thông qua Telegram phải dễ hiểu, dễ thao tác

## 3.3. Sơ đồ khối hệ thống

Hệ thống được thiết kế theo mô hình gồm các khối chức năng chính như sau:

- **ESP32:** Trung tâm xử lý, thực hiện toàn bộ logic hệ thống
- **OLED SSD1306:** Hiển thị thời gian thực
- **WiFi:** Kết nối Internet
- **NTP Server:** Cung cấp thời gian chuẩn
- **Telegram Server:** Trung gian giao tiếp giữa người dùng và hệ thống
- **Người dùng:** Gửi lệnh và nhận thông báo

Các khối này tương tác với nhau theo luồng dữ liệu hai chiều, trong đó ESP32 đóng vai trò trung tâm điều phối.



### 3.4. Mô tả hoạt động hệ thống

Hệ thống hoạt động theo các bước sau:

1. Khi khởi động, ESP32 kết nối tới mạng WiFi
2. ESP32 gửi yêu cầu đến máy chủ NTP để lấy thời gian thực
3. Thời gian nhận được được hiển thị liên tục trên màn hình OLED
4. ESP32 gửi yêu cầu đến Telegram để kiểm tra tin nhắn mới
5. Khi người dùng gửi lệnh:
  - **/time**: hệ thống trả về thời gian hiện tại
  - **/setupnoti HH:MM**: hệ thống lưu thời gian và chờ đến thời điểm tương ứng
6. Khi đến thời gian đã cài đặt, ESP32 tự động gửi thông báo qua Telegram

Quá trình này được lặp lại liên tục trong vòng lặp chính của chương trình.

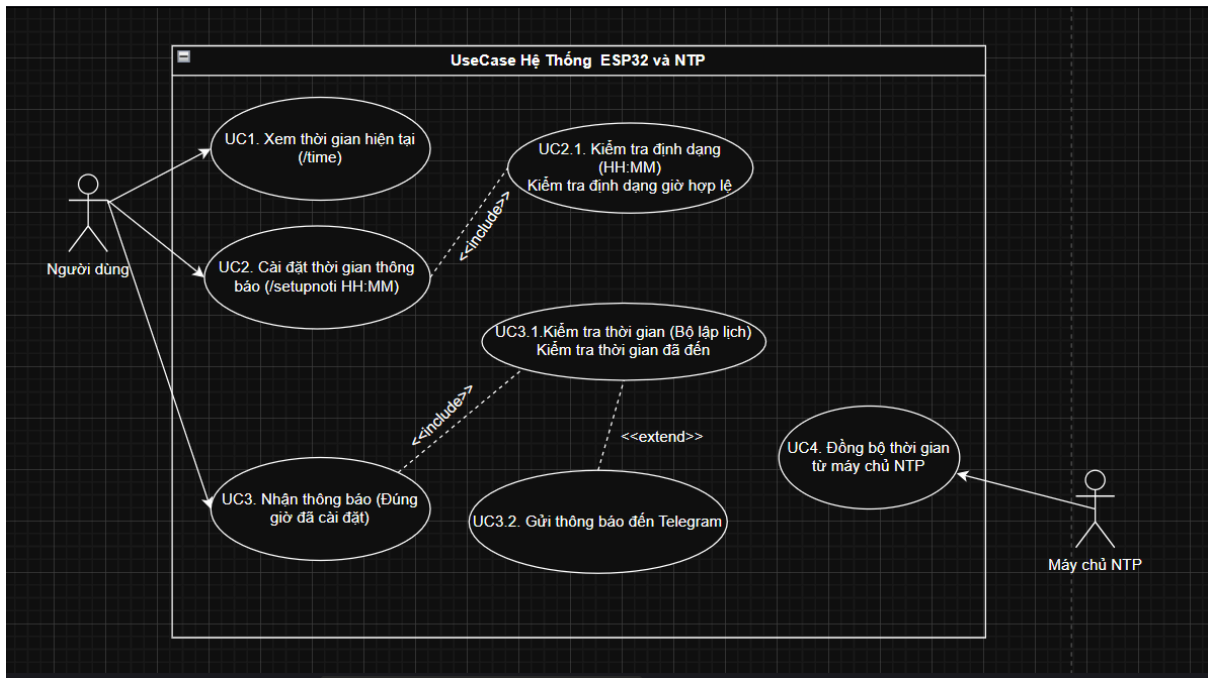
### 3.5. Use Case hệ thống

Trong hệ thống, actor chính là người dùng Telegram. Người dùng tương tác với hệ thống thông qua các lệnh.

Các use case chính:

- Xem thời gian hiện tại
- Thiết lập thời gian thông báo

Mối quan hệ giữa actor và hệ thống có thể được biểu diễn thông qua sơ đồ Use Case.



### 3.6. Kịch bản hoạt động (Scenario)

#### Kịch bản 1: Xem thời gian

- Người dùng gửi lệnh /time trong Telegram
- ESP32 nhận và xử lý lệnh
- Hệ thống lấy thời gian từ NTP
- Gửi lại thời gian hiện tại cho người dùng

#### Kịch bản 2: Đặt lịch thông báo

- Người dùng gửi lệnh /setupnoti 17:05
- ESP32 phân tích và lưu thời gian
- Hệ thống xác nhận lại với người dùng

- Khi đến 17:05, hệ thống tự động gửi thông báo

### **3.7. Tổng kết chương**

Chương này đã trình bày quá trình phân tích hệ thống, bao gồm tổng quan, yêu cầu chức năng và phi chức năng, sơ đồ khối, mô tả hoạt động và các kịch bản sử dụng.

Những nội dung này là cơ sở để tiến hành thiết kế và triển khai hệ thống trong các chương tiếp theo.

## **CHƯƠNG 4: THIẾT KẾ HỆ THỐNG**

### **4.1. Tổng quan thiết kế**

Dựa trên các yêu cầu đã phân tích ở Chương 3, hệ thống được thiết kế theo mô hình IoT gồm hai phần chính: phần cứng và phần mềm. Trong đó, phần cứng đảm nhiệm việc thu thập và hiển thị dữ liệu, còn phần mềm chịu trách nhiệm xử lý logic, kết nối mạng và giao tiếp với người dùng.

Hệ thống sử dụng vi điều khiển ESP32 làm trung tâm điều khiển, kết nối với màn hình OLED thông qua giao thức I2C, đồng thời kết nối Internet thông qua WiFi để đồng bộ thời gian từ máy chủ NTP và giao tiếp với Telegram Bot.

### **4.2. Thiết kế phần cứng**

#### **4.2.1. Danh sách linh kiện**

Các linh kiện được sử dụng trong hệ thống bao gồm:

- Vi điều khiển ESP32 DevKit
- Màn hình OLED SSD1306 (128x64)
- Dây nối (jumper wires)

Các linh kiện này có chi phí thấp, dễ sử dụng và phù hợp cho các ứng dụng IoT cơ bản.

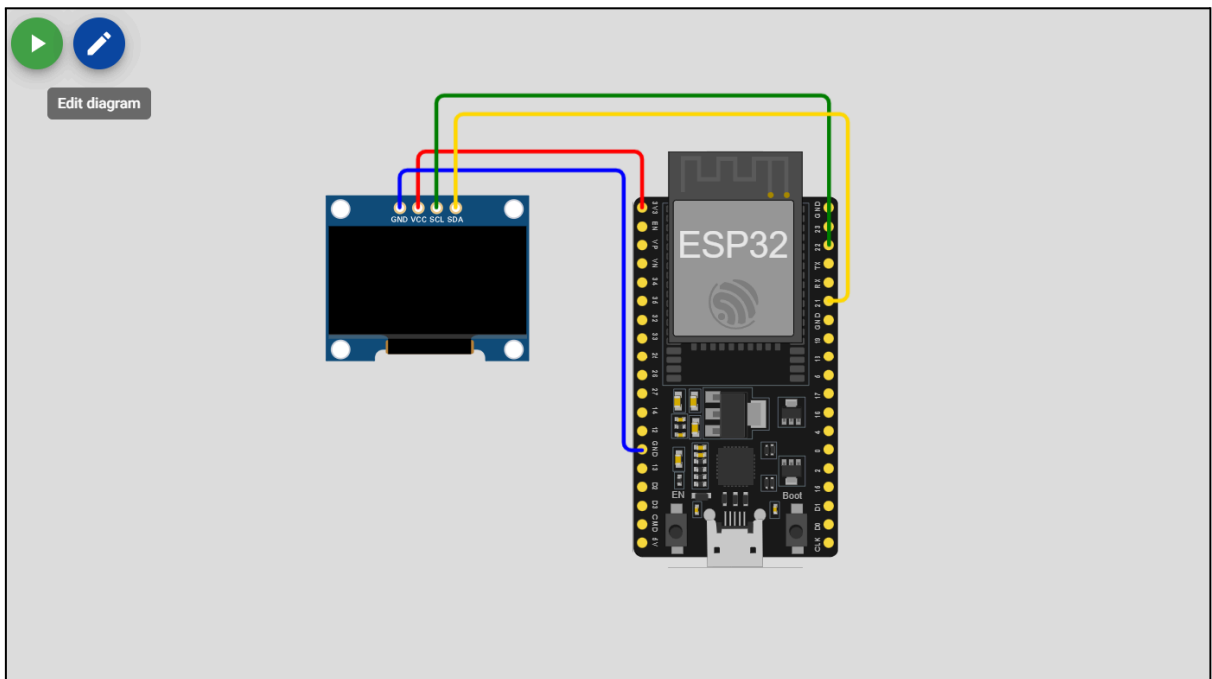
#### 4.2.2. Sơ đồ kết nối phần cứng

Việc kết nối giữa ESP32 và màn hình OLED được thực hiện thông qua giao thức I2C. Các chân kết nối cụ thể như sau:

- VCC (OLED) → 3V3 (ESP32)
- GND (OLED) → GND (ESP32)
- SDA (OLED) → GPIO21 (ESP32)
- SCL (OLED) → GPIO22 (ESP32)

Giao thức I2C giúp giảm số lượng dây kết nối, đồng thời cho phép truyền dữ liệu hiệu quả giữa ESP32 và màn hình hiển thị.

- Giao diện kết nối phần cứng





- Sơ đồ kết nối phần cứng

```
{  
  
  "version": 1,  
  
  "author": "Lộc Trần Hoàng",  
  
  "editor": "wokwi",  
  
  "parts": [  
  
    { "type": "board-esp32-devkit-c-v4", "id": "esp", "top": 0, "left": 24.04, "attrs": {} },  
  
    {  
  
      "type": "board-ssd1306",  
  
      "id": "oled1",  
  
      "top": 22.34,  
  
      "left": -134.17,  
  
      "attrs": { "i2cAddress": "0x3c" }  
  
    },  
  
  ],  
  
  "connections": [  
  
    [ "esp:TX", "$SerialMonitor:RX", "", [] ],  
  
    [ "esp:RX", "$SerialMonitor:TX", "", [] ],  
  
    [ "oled1:VCC", "esp:3V3", "red", [ "v-28.8", "h105.6" ] ],  
  
    [ "oled1:GND", "esp:GND.1", "blue", [ "v-19.2", "h115.2", "v144" ] ],  
  
    [ "esp:22", "oled1:SCL", "green", [ "v-76.8", "h-201.9" ] ],  
  
    [ "oled1:SDA", "esp:21", "gold", [ "v-48", "h201.67", "v96" ] ]  
  
  ],  
  
}
```

```
"dependencies": {}
```

### 4.2.3. Nguyên lý hoạt động phần cứng

ESP32 đóng vai trò cung cấp nguồn và điều khiển toàn bộ hệ thống. Sau khi được cấp nguồn, ESP32 sẽ thực hiện kết nối với mạng WiFi và bắt đầu hoạt động.

Màn hình OLED nhận dữ liệu từ ESP32 thông qua giao thức I2C và hiển thị thông tin thời gian theo định dạng giờ, phút, giây. Việc hiển thị được cập nhật liên tục theo thời gian thực.

## 4.3. Thiết kế phần mềm

### 4.3.1. Kiến trúc chương trình

Chương trình được xây dựng theo mô hình vòng lặp chính gồm hai phần:

- **Hàm setup():**
  - Khởi tạo Serial
  - Kết nối WiFi
  - Khởi tạo màn hình OLED
  - Khởi tạo NTP Client
  - Gửi thông báo khởi động hệ thống
- **Hàm loop():**
  - Cập nhật thời gian từ NTP
  - Hiển thị thời gian lên OLED
  - Kiểm tra và xử lý lệnh từ Telegram
  - Kiểm tra điều kiện gửi thông báo

Ngoài ra, chương trình còn sử dụng các hàm hỗ trợ như:

- `sendTelegram()`: gửi tin nhắn

- checkTelegram(): nhận và xử lý lệnh

#### **4.3.2. Thiết kế các chức năng chính**

##### **a) Kết nối WiFi**

ESP32 sử dụng thư viện WiFi để kết nối tới mạng không dây. Sau khi kết nối thành công, thiết bị có thể truy cập Internet để sử dụng các dịch vụ như NTP và Telegram.

##### **b) Đồng bộ thời gian NTP**

Hệ thống sử dụng thư viện NTPClient để lấy thời gian từ máy chủ NTP. Thời gian được cập nhật định kỳ và điều chỉnh theo múi giờ GMT+7.

Việc sử dụng NTP giúp đảm bảo thời gian chính xác mà không cần phần cứng RTC.

##### **c) Hiển thị thời gian trên OLED**

Thời gian sau khi được đồng bộ sẽ được hiển thị trên màn hình OLED dưới dạng:

HH:MM:SS

Việc hiển thị được cập nhật liên tục mỗi giây, giúp người dùng dễ dàng quan sát.

##### **d) Giao tiếp với Telegram**

ESP32 sử dụng giao thức HTTPS để giao tiếp với Telegram Bot API. Hai chức năng chính bao gồm:

- Gửi tin nhắn (sendMessage)
- Nhận tin nhắn (getUpdates)

Việc giao tiếp này cho phép hệ thống nhận lệnh từ người dùng và gửi phản hồi tương ứng.

##### **e) Xử lý lệnh từ người dùng**

Hệ thống hỗ trợ hai lệnh chính:

- **/time:** Trả về thời gian hiện tại
- **/setupnoti HH:MM:** Thiết lập thời gian gửi thông báo

ESP32 sẽ phân tích nội dung tin nhắn, trích xuất thông tin cần thiết và thực hiện xử lý tương ứng.

#### f) Logic gửi thông báo

Sau khi người dùng thiết lập thời gian, hệ thống sẽ lưu giá trị giờ và phút. Trong mỗi vòng lặp, ESP32 sẽ so sánh thời gian hiện tại với thời gian đã cài đặt.

Khi thời gian trùng khớp, hệ thống sẽ tự động gửi thông báo qua Telegram và đánh dấu đã gửi để tránh lặp lại.

#### 4.3.3. Chương trình điều khiển

```
#include <WiFi.h>

#include <WiFiClientSecure.h>

#include <Wire.h>

#include <NTPTClient.h>

#include <WiFiUdp.h>

#include <Adafruit_SSD1306.h>

#include <Adafruit_GFX.h>

// WiFi

const char* ssid = "Wokwi-GUEST";

const char* password = "";

// Telegram
```

```

String BOT_TOKEN =
"8736947766:AAECk-uGtVkp3PHAKdyh9uNiis44k-6wTI";

String CHAT_ID = "-5160714911";

WiFiClientSecure client;


// OLED

#define SCREEN_WIDTH 128

#define SCREEN_HEIGHT 64

#define OLED_RESET -1

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire,
OLED_RESET);


// NTP

WiFiUDP ntpUDP;

NTPClient timeClient(ntpUDP, "pool.ntp.org", 7 * 3600, 60000);


// Update tracking

long lastUpdateId = 0;


// Alarm

int alarmHour = -1;

int alarmMinute = -1;

bool alarmTriggered = false;

```

```

// -----

// Telegram

// -----

void sendTelegram(String message) {

    client.setInsecure();

    message.replace(" ", "%20");

    String url = "/bot" + BOT_TOKEN + "/sendMessage?chat_id=" +
CHAT_ID + "&text=" + message;

    if (client.connect("api.telegram.org", 443)) {

        client.println("GET " + url + " HTTP/1.1");

        client.println("Host: api.telegram.org");

        client.println("Connection: close");

        client.println();

    }

}

// -----

// Telegram

// -----

void checkTelegram() {

    client.setInsecure();

```

```

    String url = "/bot" + BOT_TOKEN + "/getUpdates?offset=" +
String(lastUpdateId + 1);

    if (!client.connect("api.telegram.org", 443)) {

        Serial.println("GetUpdates failed");

        return;

    }

    client.println("GET " + url + " HTTP/1.1");

    client.println("Host: api.telegram.org");

    client.println("Connection: close");

    client.println();

    String response = "";

    while (client.connected()) {

        response += client.readString();

    }

    Serial.println(response);

    // -----

    // /time

    // -----

    if (response.indexOf("/time") != -1) {

        Serial.println("Command /time detected");

```

```

timeClient.update();

int h = timeClient.getHours();

int m = timeClient.getMinutes();

int s = timeClient.getSeconds();

String msg = "Time: ";

msg += (h < 10 ? "0" : "") + String(h) + ":";

msg += (m < 10 ? "0" : "") + String(m) + ":";

msg += (s < 10 ? "0" : "") + String(s);

sendTelegram(msg);

}

// -----
// /setupnoti HH:MM
// -----

int setIndex = response.indexOf("/setupnoti");

if (setIndex != -1) {

    Serial.println("Command /setupnoti detected");

    int spaceIndex = response.indexOf(" ", setIndex);

    int colonIndex = response.indexOf(":", setIndex);

```



```

        if (spaceIndex != -1 && colonIndex != -1) {

            String hourStr = response.substring(spaceIndex + 1,
colonIndex);

            String minStr = response.substring(colonIndex + 1,
colonIndex + 3);

            alarmHour = hourStr.toInt();

            alarmMinute = minStr.toInt();

            alarmTriggered = false;

            Serial.println("Hour: " + String(alarmHour));

            Serial.println("Minute: " + String(alarmMinute));

            String confirm = "Da dat thong bao luc ";

            confirm += (alarmHour < 10 ? "0" : "") + String(alarmHour) +
":";

            confirm += (alarmMinute < 10 ? "0" : "") +
String(alarmMinute);

            sendTelegram(confirm);

        } else {

            sendTelegram("Sai dinh dang! Dung: /setupnoti HH:MM");

        }

    }
}

```

```

// update_id

int idx = response.lastIndexOf("\"update_id\":");

if (idx != -1) {

    String sub = response.substring(idx + 12);

    lastUpdateId = sub.toInt();

}

}

// -----

// Setup

// -----

void setup() {

    Serial.begin(115200);

    WiFi.begin(ssid, password);

    while (WiFi.status() != WL_CONNECTED) {

        delay(500);

        Serial.print(".");

    }

    Serial.println("\nWiFi Connected");

    Wire.begin(21, 22);

```

```

display.begin(SSD1306_SWITCHCAPVCC, 0x3C);

display.setTextSize(2);

display.setTextColor(SSD1306_WHITE);


timeClient.begin();


// Gửi hướng dẫn

String guide =

    "HUONG DAN SU DUNG:%0A"

    "/time - Xem thoi gian hien tai%0A"

    "/setupnoti HH:MM - Dat thong bao";

sendTelegram(guide);
}

// -----

// Loop

// -----

void loop() {

    timeClient.update();


    int h = timeClient.getHours();

    int m = timeClient.getMinutes();

    int s = timeClient.getSeconds();

```

```

// OLED

display.clearDisplay();

display.setCursor(0, 0);

display.printf("%02d:%02d:%02d", h, m, s);

display.display();


// Check Telegram

checkTelegram();


// -----

// CHECK ALARM

// -----

if (alarmHour != -1 && alarmMinute != -1) {

    if (h == alarmHour && m == alarmMinute && !alarmTriggered) {

        String msg = "Bay gio la ";

        msg += (h < 10 ? "0" : "") + String(h) + ":";

        msg += (m < 10 ? "0" : "") + String(m);

        sendTelegram(msg);

        alarmTriggered = true;

    }
}

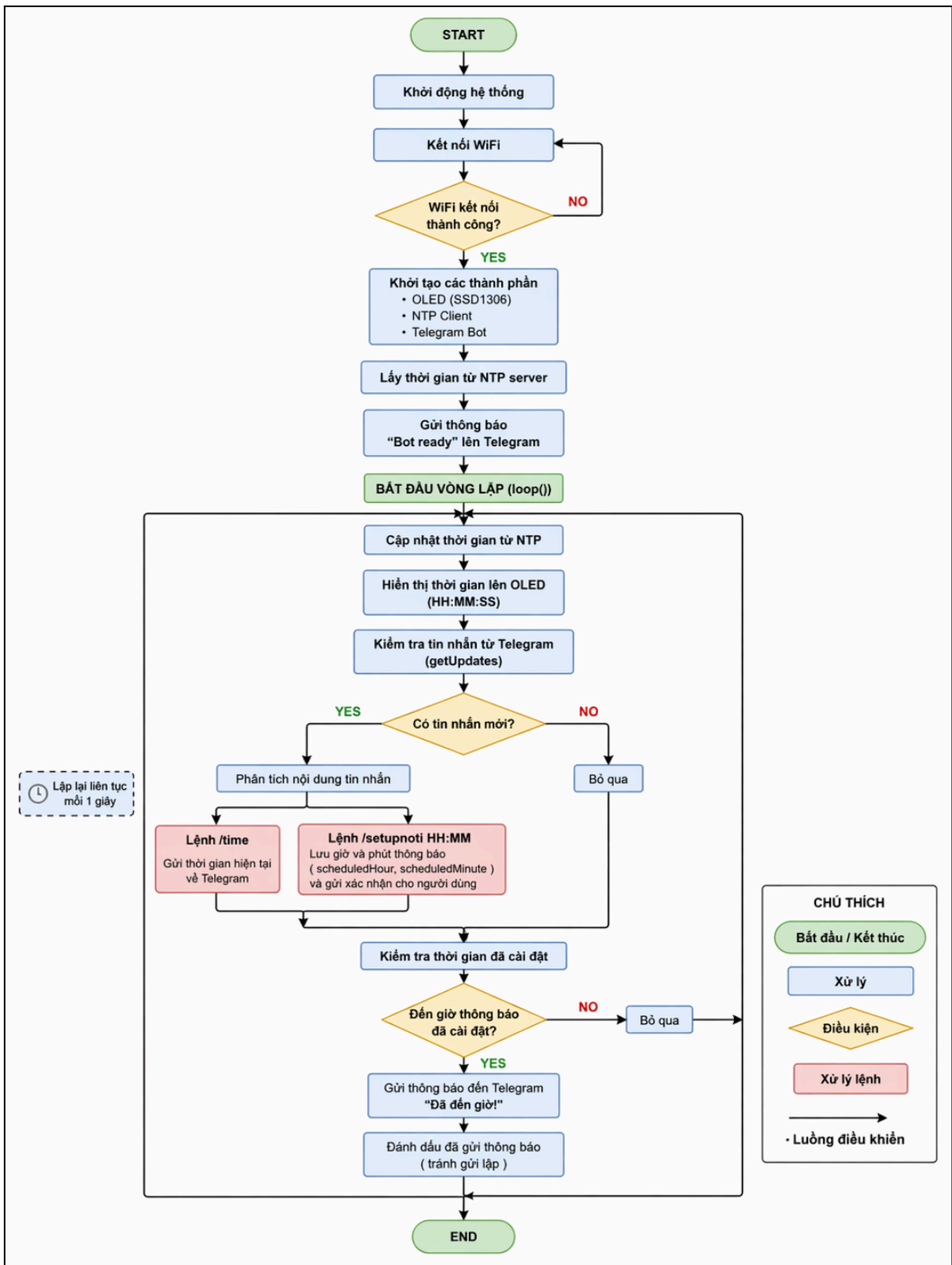
```

```
}  
  
delay(3000);}
```

## 4.4. Lưu đồ thuật toán

Hoạt động của hệ thống có thể được mô tả thông qua lưu đồ thuật toán như sau:

1. Bắt đầu
2. Kết nối WiFi
3. Lấy thời gian từ NTP
4. Hiển thị lên OLED
5. Kiểm tra tin nhắn Telegram
6. Xử lý lệnh (nếu có)
7. Kiểm tra điều kiện gửi thông báo
8. Lặp lại vòng lặp



## 4.5. Tổng kết chương

Chương này đã trình bày chi tiết quá trình thiết kế hệ thống, bao gồm thiết kế phần cứng và phần mềm. Các thành phần của hệ thống được xây dựng một cách hợp lý, đảm bảo đáp ứng các yêu cầu đã đề ra.

Thiết kế này là cơ sở để triển khai và kiểm thử hệ thống trong chương tiếp theo.

# CHƯƠNG 5: TRIỂN KHAI VÀ KẾT QUẢ

## 5.1. Môi trường phát triển

Hệ thống được triển khai trong môi trường giả lập kết hợp với công cụ lập trình hiện đại, giúp quá trình phát triển và kiểm thử trở nên thuận tiện.

Các công cụ và nền tảng được sử dụng bao gồm:

- **PlatformIO:** Môi trường phát triển tích hợp (IDE) hỗ trợ lập trình cho ESP32
- **Wokwi Simulator:** Công cụ giả lập phần cứng, cho phép mô phỏng mạch ESP32 và OLED
- **Arduino Framework:** Thư viện hỗ trợ lập trình cho vi điều khiển ESP32

Ngoài ra, hệ thống sử dụng các thư viện sau:

- WiFi
- NTPClient
- Adafruit SSD1306
- Adafruit GFX

## 5.2. Triển khai hệ thống

### 5.2.1. Kết nối WiFi

Hệ thống sử dụng thư viện WiFi để kết nối tới mạng không dây. Sau khi khởi động, ESP32 thực hiện kết nối tới WiFi và kiểm tra trạng thái kết nối.

Khi kết nối thành công, hệ thống hiển thị thông báo trên Serial Monitor.

```
load:0x3fff0030,len:1156
load:0x40078000,len:11456
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc
..
WiFi Connected
[ 8575][E][ssl_client.cpp:37] _handle_error(): [send_ssl_data():382]: (-27136) SSL - A buffer is too small to receive or
write a message
```

### 5.2.2. Đồng bộ thời gian NTP

Sau khi kết nối Internet, ESP32 sử dụng giao thức NTP để đồng bộ thời gian từ máy chủ. Thời gian được cập nhật định kỳ và điều chỉnh theo múi giờ GMT+7.

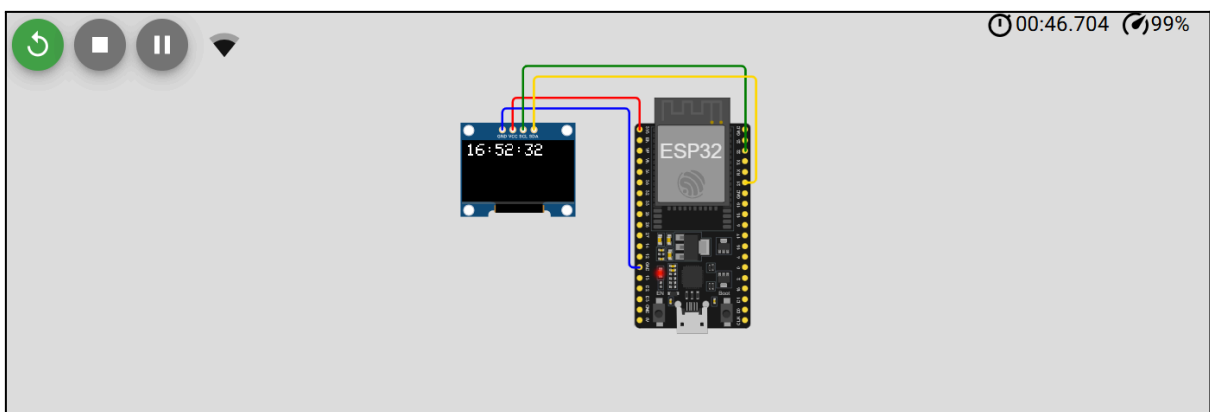
Việc đồng bộ thời gian giúp đảm bảo hệ thống luôn hoạt động với thời gian chính xác.

### 5.2.3. Hiển thị thời gian trên OLED

Thời gian nhận được từ NTP được hiển thị trên màn hình OLED theo định dạng:

HH:MM:SS

Màn hình được cập nhật liên tục mỗi giây, giúp người dùng quan sát thời gian thực một cách trực quan.

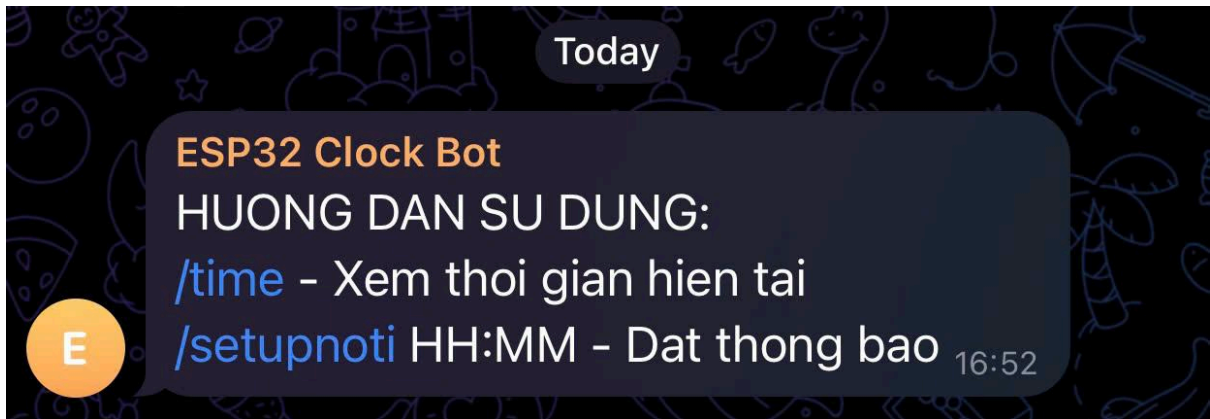


### 5.2.4. Tích hợp Telegram Bot



Hệ thống tích hợp Telegram Bot để giao tiếp với người dùng thông qua Internet. ESP32 gửi và nhận dữ liệu bằng giao thức HTTPS thông qua Telegram Bot API.

Sau khi hệ thống khởi động, bot sẽ gửi một tin nhắn thông báo sẵn sàng hoạt động kèm theo hướng dẫn sử dụng.



#### 5.2.5. Xử lý lệnh từ người dùng

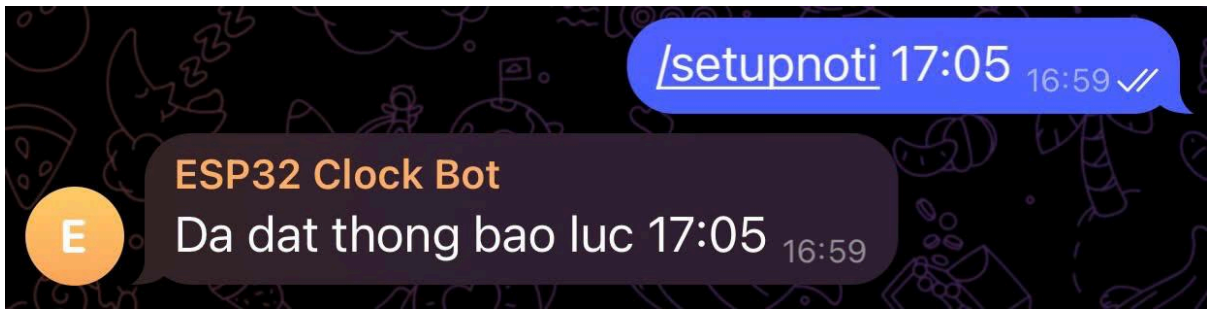
Hệ thống hỗ trợ hai lệnh chính:

- **/time:** Trả về thời gian hiện tại
- **/setupnoti HH:MM:** Thiết lập thời gian gửi thông báo

Khi người dùng gửi lệnh **/time**, hệ thống sẽ lấy thời gian hiện tại và gửi lại thông qua Telegram.



Khi người dùng gửi lệnh **/setupnoti**, hệ thống sẽ phân tích chuỗi thời gian, lưu giá trị giờ và phút, sau đó gửi lại thông báo xác nhận.



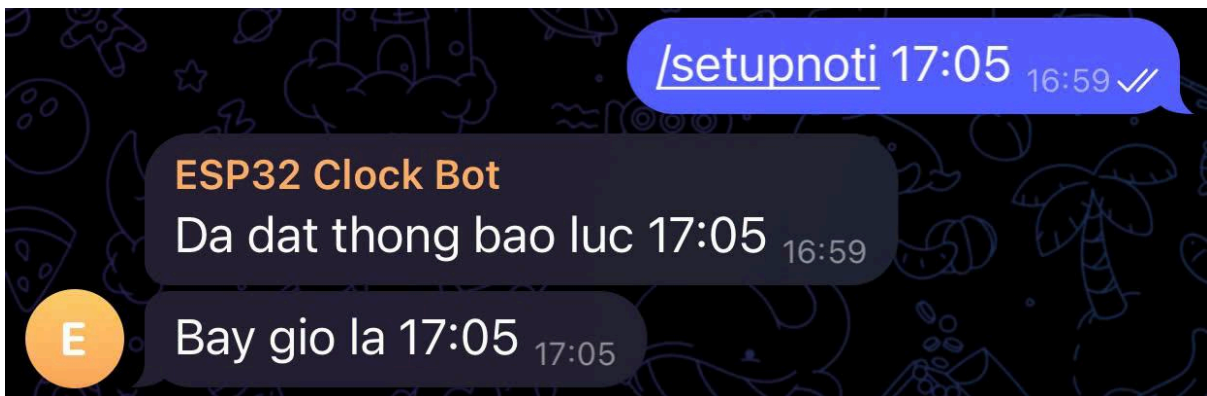
### 5.2.6. Chức năng thông báo tự động

Sau khi thiết lập thời gian, hệ thống sẽ kiểm tra thời gian trong mỗi vòng lặp. Khi thời gian hiện tại trùng với thời gian đã cài đặt, ESP32 sẽ tự động gửi thông báo qua Telegram.

Thông báo được gửi dưới dạng:

“Bây giờ là HH:MM”

Để tránh gửi lặp lại, hệ thống sử dụng biến trạng thái để đánh dấu đã gửi thông báo.



### 5.3. Kết quả đạt được

Sau quá trình triển khai và kiểm thử, hệ thống đã đạt được các kết quả sau:

- Kết nối WiFi thành công và ổn định
- Đồng bộ thời gian chính xác từ máy chủ NTP
- Hiển thị thời gian liên tục trên màn hình OLED
- Telegram Bot hoạt động ổn định
- Xử lý đúng các lệnh từ người dùng

- Thiết lập và gửi thông báo theo lịch trình thành công

Hệ thống hoạt động đúng theo yêu cầu đề ra và đáp ứng đầy đủ các chức năng đã thiết kế.

## **5.4. Đánh giá hệ thống**

### **5.4.1. Ưu điểm**

- Hệ thống đơn giản, dễ triển khai
- Sử dụng NTP giúp thời gian chính xác
- Giao diện điều khiển thông qua Telegram tiện lợi
- Hoạt động theo thời gian thực
- Dễ mở rộng và nâng cấp

### **5.4.2. Hạn chế**

- Chỉ hỗ trợ một mốc thời gian thông báo
- Không lưu dữ liệu khi mất điện
- Việc xử lý chuỗi lệnh còn đơn giản
- Chưa có cơ chế bảo mật nâng cao

## **5.5. Hướng phát triển**

Trong tương lai, hệ thống có thể được cải tiến theo các hướng sau:

- Hỗ trợ nhiều lịch thông báo khác nhau
- Lưu dữ liệu vào bộ nhớ EEPROM hoặc Flash
- Xây dựng ứng dụng di động để điều khiển
- Phát triển giao diện web (Web Dashboard)
- Tích hợp thêm cảm biến để mở rộng chức năng

## 5.6. Tổng kết chương

Chương này đã trình bày quá trình triển khai hệ thống từ môi trường phát triển đến các chức năng cụ thể, đồng thời đánh giá kết quả đạt được.

Các kết quả cho thấy hệ thống hoạt động ổn định và đáp ứng tốt yêu cầu của đề tài. Đây là cơ sở để tổng kết và kết luận trong chương tiếp theo.

# CHƯƠNG 6: KẾT LUẬN

## 6.1. Kết luận

Trong khuôn khổ đề tài “Xây dựng đồng hồ thời gian thực với ESP32 và NTP”, hệ thống đã được thiết kế, triển khai và kiểm thử thành công. Thông qua quá trình thực hiện, đề tài đã đạt được các mục tiêu đề ra ban đầu.

Cụ thể, hệ thống đã thực hiện được các chức năng chính như:

- Kết nối WiFi và truy cập Internet
- Đồng bộ thời gian chính xác từ máy chủ NTP
- Hiển thị thời gian thực trên màn hình OLED
- Xây dựng và tích hợp Telegram Bot để giao tiếp với người dùng
- Xử lý lệnh `/time` để cung cấp thời gian hiện tại
- Cho phép thiết lập lịch thông báo thông qua lệnh `/setupnoti`
- Gửi thông báo tự động qua Telegram khi đến thời điểm đã cài đặt

Kết quả thực nghiệm cho thấy hệ thống hoạt động ổn định, thời gian đồng bộ chính xác và khả năng tương tác với người dùng thông qua Telegram đạt hiệu quả tốt. Điều này chứng minh tính khả thi của việc ứng dụng các công nghệ IoT trong việc xây dựng các hệ thống thông minh, đơn giản nhưng hữu ích.

Ngoài ra, quá trình thực hiện đề tài cũng giúp củng cố và nâng cao kiến thức về lập trình nhúng, giao tiếp mạng, xử lý dữ liệu và tích hợp các dịch vụ bên ngoài. Đây là những kỹ năng quan trọng trong lĩnh vực phát triển hệ thống IoT hiện nay.

## 6.2. Hướng phát triển

Mặc dù hệ thống đã đáp ứng được các yêu cầu cơ bản, vẫn còn nhiều hướng phát triển để nâng cao tính năng và hiệu quả hoạt động:

- Mở rộng hệ thống để hỗ trợ nhiều lịch thông báo khác nhau
- Lưu trữ dữ liệu cấu hình bằng EEPROM hoặc bộ nhớ Flash để duy trì sau khi mất điện
- Phát triển giao diện người dùng trực quan hơn thông qua ứng dụng di động hoặc web
- Tích hợp thêm các cảm biến (nhiệt độ, độ ẩm, ánh sáng) để mở rộng chức năng
- Nâng cao khả năng bảo mật trong giao tiếp mạng
- Tối ưu hiệu năng và giảm độ trễ trong xử lý dữ liệu

Trong tương lai, hệ thống có thể được phát triển thành một nền tảng IoT hoàn chỉnh, phục vụ cho các ứng dụng thực tế như nhà thông minh, hệ thống nhắc việc tự động hoặc giám sát thiết bị từ xa.

## 6.3. Tổng kết

Đề tài đã hoàn thành việc xây dựng một hệ thống IoT cơ bản nhưng đầy đủ chức năng, kết hợp giữa phần cứng và phần mềm, giữa thiết bị và dịch vụ mạng.

Những kết quả đạt được không chỉ có ý nghĩa trong học tập mà còn có tiềm năng ứng dụng trong thực tế. Đây là nền tảng quan trọng để tiếp tục nghiên cứu và phát triển các hệ thống IoT phức tạp hơn trong tương lai.

# Tài liệu tham khảo

- [1] Viettel IDC (2024). IoT là gì? Lợi ích và ứng dụng của Internet of Things.  
<https://viettelidc.com.vn/tin-tuc/iot-la-gi-nhung-ung-dung-noi-bat-cua-iot>
- [2] Mescu.vn (2023). Giới thiệu về bo mạch phát triển ESP32  
<https://mescu.vn/ho-tro-ky-thuat/gioi-thieu-ve-bo-mach-phat-trien-esp32.KyJ>
- [3] linhkienx (2025). Phân tích giao tiếp I2C & SPI trên OLED – Khi tốc độ hiển thị quyết định trải nghiệm  
<https://linhkienx.com/tin-tuc/phan-tich-giao-tiep-i2c-spi-tren-oled-khi-toc-do-hien-thi-quyet-dinh-trai-nghiem>
- [5] VIBLO (2017). Tổng quan về giao thức Network Time Protocol (NTP)  
<https://viblo.asia/p/tong-quan-ve-giao-thuc-network-time-protocol-ntp-6J3Zg20AKmB>
- [6] randomnerdtutorials. ESP32 NTP Client-Server: Get Date and Time (Arduino IDE)  
<https://randomnerdtutorials.com/esp32-date-time-ntp-client-server-arduino/>
- [7] fptshop (2025). Bot Telegram là gì? Hướng dẫn cách tạo và lập trình Bot Telegram chi tiết, dễ thực hiện  
<https://fptshop.com.vn/tin-tuc/thu-thuat/bot-telegram-la-gi-162528>
- [8] electronicwings. Send a Telegram message using ESP32  
<https://www.electronicwings.com/esp32/send-a-telegram-message-using-esp32>

